



Estácio Unidade Nova Friburgo - RJ

Desenvolvimento Full Stack

Disciplina: DGT2821

Turma: 9001

Semestre: 2026.2

RELATÓRIO DE PRÁTICAS

Criação das Entidades e Sistema de Persistência

Aluno: Ana Carolina Portela Siqueira

Matrícula: 20206073067

Github: <https://github.com/itsmesiq/CadastroPOO>

Nova Friburgo — RJ

2026

Contents

1	Introdução	2
2	Objetivos	2
2.1	Objetivo Geral	2
3	Desenvolvimento	2
3.1	Classe Pessoa	2
3.1.1	Classe PessoaFisica	3
3.1.2	Classe PessoaJuridica	4
3.1.3	Repositório	5
3.2	Main e Resultados da Execução	8
3.3	Análise e Conclusão	9
3.3.1	Quais as vantagens e desvantagens do uso de herança?	9
3.3.2	Por que a interface Serializable é necessária ao efetuar persistência em arquivos binários?	10
3.3.3	Como o paradigma funcional é utilizado pela API Stream no Java?	10
3.3.4	Quando trabalhamos com Java, qual padrão de desenvolvimento é adotado na persistência de dados em arquivos?	10

1 Introdução

Este relatório apresenta o desenvolvimento do primeiro procedimento da prática de Programação Orientada a Objetos, voltado à implementação das Entidades e Sistemas de Persistência de um sistema de cadastro de pessoas.

Nesta etapa, foi desenvolvida as classes Pessoa, PessoaFisica, PessoaJurica e os repositórios PessoaFisicaRepo e PessoaJuridicaRepo.

O desenvolvimento do procedimento permitiu aplicar, de forma prática, conceitos de Programação Orientada a Objetos, como herança, encapsulamento, criação e utilização de objetos, além da separação das responsabilidades entre as entidades e os respectivos repositórios.

2 Objetivos

2.1 Objetivo Geral

Desenvolver uma aplicação Java orientada a objetos para cadastro de pessoas físicas e jurídicas, utilizando os conceitos de classes, herança, polimorfismo, encapsulamento e interfaces, além de implementar o armazenamento e a recuperação dos objetos em arquivos binários por meio de serialização e repositórios baseados em ArrayList para o gerenciamento das entidades.

3 Desenvolvimento

Inicialmente, foi criado o projeto CadastroPOO, utilizando Java e Apache Ant para compilação e execução da aplicação. As entidades foram organizadas no pacote model:

3.1 Classe Pessoa

A classe Pessoa foi utilizada como classe-base da aplicação, contendo os atributos:

- id
- nome
- email

Foram implementados construtor, getters, setters e o método exibir().

A classe também implementa a interface Serializable, permitindo que os objetos possam ser serializados e armazenados em arquivos binários. Como as demais classes criadas herdam de Pessoa, essa interface foi implementada somente nessa classe.

Visando reaproveitamento do código e evitando repetir trechos desnecessários.

```
package model;
import java.io.Serializable;

public class Pessoa implements Serializable {
    private int id;
    private String nome;
    private String email;

    // Constructor
    public Pessoa(int id, String nome, String email) {
        this.id = id;
        this.nome = nome;
        this.email = email;
    }

    // Getters
    public int getId() {
        return id;
    }

    public String getNome() {
        return nome;
    }

    public String getEmail() {
        return email;
    }

    // Setters
    public void setId(int id) {
        this.id = id;
    }

    public void setNome(String nome) {
        this.nome = nome;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public void exibir() {
        System.out.println("ID: " + id);
        System.out.println("Nome: " + nome);
        System.out.println("Email: " + email);
    }
}
```

Figura 1 – Classe Pessoa

3.1.1 Classe PessoaFisica

Além dos atributos herdados de Pessoa, foram adicionados:

- cpf
- idade

A classe possui construtor, getters, setters e sobrescreve o método `exibir()`. Utilizando `super.exibir()` para reaproveitar o comportamento definido na classe Pessoa.

```
package model;

public class PessoaFisica extends Pessoa {
    private String cpf;
    private int idade;

    // Constructor
    public PessoaFisica(int id, String nome, String email, String cpf, int idade) {
        super(id, nome, email);
        this.cpf = cpf;
        this.idade = idade;
    }

    // Getters
    public String getCpf() {
        return cpf;
    }

    public int getIdade() {
        return idade;
    }

    // Setters
    public void setCpf(String cpf) {
        this.cpf = cpf;
    }

    public void setIdade(int idade) {
        this.idade = idade;
    }

    @Override
    public void exibir() {
        super.exibir();
        System.out.println("CPF: " + cpf);
        System.out.println("Idade: " + idade);
    }
}
```

Figura 2 – Classe PessoaFisica

3.1.2 Classe PessoaJuridica

Além dos atributos herdados de Pessoa, foi adicionado:

- cnpj A classe possui construtor, getters, setters e sobreescreve o método exibir(). Utilizando super.exibir() para reaproveitar o comportamento definido na classe Pessoa.

```
package model;

public class PessoaJuridica extends Pessoa {
    private String cnpj;

    public PessoaJuridica(int id, String nome, String email, String cnpj) {
        super(id, nome, email);
        this.cnpj = cnpj;
    }

    // Getters
    public String getCnpj () {
        return cnpj;
    }

    // Setters
    public void setCnpj(String cnpj){
        this.cnpj = cnpj;
    }

    @Override
    public void exibir() {
        super.exibir();
        System.out.println("CNPJ: " + cnpj);
    }
}
```

Figura 3 – Classe PessoaJuridica

3.1.3 Repositório

Foram criados os repositórios: PessoaFisicaRepo e PessoaJuridicaRepo. Cada repositório mantém um ArrayList com suas respectivas entidades e implementa as seguintes operações:

- Inserir
- Obter Todos
- Obter
- Excluir
- Alterar
- Persistir
- Recuperar

```
package model;

import java.io.IOException;
import java.util.ArrayList;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;

public class PessoaJuridicaRepo {
    private ArrayList<PessoaJuridica> pessoasJuridicas = new ArrayList<>();

    public void inserir(PessoaJuridica pessoaJuridica) {
        pessoasJuridicas.add(pessoaJuridica);
    }

    public ArrayList<PessoaJuridica> obterTodos () {
        return pessoasJuridicas;
    }

    public PessoaJuridica obter(int id) {
        for (PessoaJuridica pessoaJuridica : pessoasJuridicas) {
            if (pessoaJuridica.getId() == id) {
                return pessoaJuridica;
            }
        }
        return null;
    }

    public void excluir(int id) {
        pessoasJuridicas.removeIf(pessoaJuridica -> pessoaJuridica.getId() == id);
    }

    public void alterar(PessoaJuridica pessoaJuridica) {
        PessoaJuridica pessoaExistente = obter(pessoaJuridica.getId());

        if (pessoaExistente != null) {
            pessoaExistente.setNome(pessoaJuridica.getNome());
            pessoaExistente.setEmail(pessoaJuridica.getEmail());
            pessoaExistente.setCnpj(pessoaJuridica.getCnpj());
        }
    }

    public void persistir(String nomeArquivo) throws IOException {
        try {
            FileOutputStream arquivo = new FileOutputStream(nomeArquivo);
            ObjectOutputStream objeto = new ObjectOutputStream(arquivo)
        } {
            objeto.writeObject(pessoasJuridicas);
        }
    }

    public void recuperar(String nomeArquivo) throws IOException, ClassNotFoundException {
        try {
            FileInputStream arquivo = new FileInputStream(nomeArquivo);
            ObjectInputStream objeto = new ObjectInputStream(arquivo)
        } {
            pessoasJuridicas = (ArrayList<PessoaJuridica>) objeto.readObject();
        }
    }
}
```

Figura 4 – Classe PessoaJuridicaRepo

```
package model;

import java.io.IOException;
import java.util.ArrayList;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.ObjectInputStream;
import java.io.ObjectOutputStream;

public class PessoaFisicaRepo {
    private ArrayList<PessoaFisica> pessoasFisicas = new ArrayList<>();

    public void inserir(PessoaFisica pessoaFisica) {
        pessoasFisicas.add(pessoaFisica);
    }

    public ArrayList<PessoaFisica> obterTodos () {
        return pessoasFisicas;
    }

    public PessoaFisica obter(int id) {
        for (PessoaFisica pessoaFisica : pessoasFisicas) {
            if (pessoaFisica.getId() == id) {
                return pessoaFisica;
            }
        }
        return null;
    }

    public void excluir(int id) {
        pessoasFisicas.removeIf(pessoaFisica -> pessoaFisica.getId() == id);
    }

    public void alterar(PessoaFisica pessoaFisica) {
        PessoaFisica pessoaExistente = obter(pessoaFisica.getId());

        if (pessoaExistente != null) {
            pessoaExistente.setNome(pessoaFisica.getNome());
            pessoaExistente.setEmail(pessoaFisica.getEmail());
            pessoaExistente.setCpf(pessoaFisica.getCpf());
            pessoaExistente.setIdade(pessoaFisica.getIdade());
        }
    }

    public void persistir(String nomeArquivo) throws IOException {
        try{
            FileOutputStream arquivo = new FileOutputStream(nomeArquivo);
            ObjectOutputStream objeto = new ObjectOutputStream(arquivo)
        } {
            objeto.writeObject(pessoasFisicas);
        }
    }

    public void recuperar(String nomeArquivo) throws IOException, ClassNotFoundException {
        try{
            FileInputStream arquivo = new FileInputStream(nomeArquivo);
            ObjectInputStream objeto = new ObjectInputStream(arquivo)
        } {
            pessoasFisicas = (ArrayList<PessoaFisica>) objeto.readObject();
        }
    }
}
```

Figura 5 – Classe PessoaFisicaRepo

3.2 Main e Resultados da Execução

```
import model.PessoaFisica;
import model.PessoaFisicaRepo;
import model.PessoaJuridica;
import model.PessoaJuridicaRepo;

public class Main {
    public static void main(String[] args) {
        try {
            PessoaFisicaRepo repo1 = new PessoaFisicaRepo();

            PessoaFisica pessoa1 = new PessoaFisica(1, "Archie", "archie@gmail.com", "123.456.789-00", 30);
            PessoaFisica pessoa2 = new PessoaFisica(2, "Betty", "betty@gmail.com", "987.654.321-00", 28);

            repo1.inserir(pessoa1);
            repo1.inserir(pessoa2);

            repo1.persistir("pessoas-fisicas.bin");

            System.out.println("Dados de Pessoa Física Armazenados.");

            PessoaFisicaRepo repo2 = new PessoaFisicaRepo();

            repo2.recuperar("pessoas-fisicas.bin");

            System.out.println("Dados de Pessoa Física Recuperados.");

            for(PessoaFisica pessoa : repo2.obterTodos()) {
                pessoa.exibir();
            }

            PessoaJuridicaRepo repo3 = new PessoaJuridicaRepo();

            PessoaJuridica empresa1 = new PessoaJuridica(1, "Empresa A", "empresaA@gmail.com", "12.345.678/0001-90");
            PessoaJuridica empresa2 = new PessoaJuridica(2, "Empresa B", "empresaB@gmail.com", "98.765.432/0001-90");

            repo3.inserir(empresa1);
            repo3.inserir(empresa2);

            repo3.persistir("pessoas-juridicas.bin");

            System.out.println("Dados de Pessoa Jurídica Armazenados.");

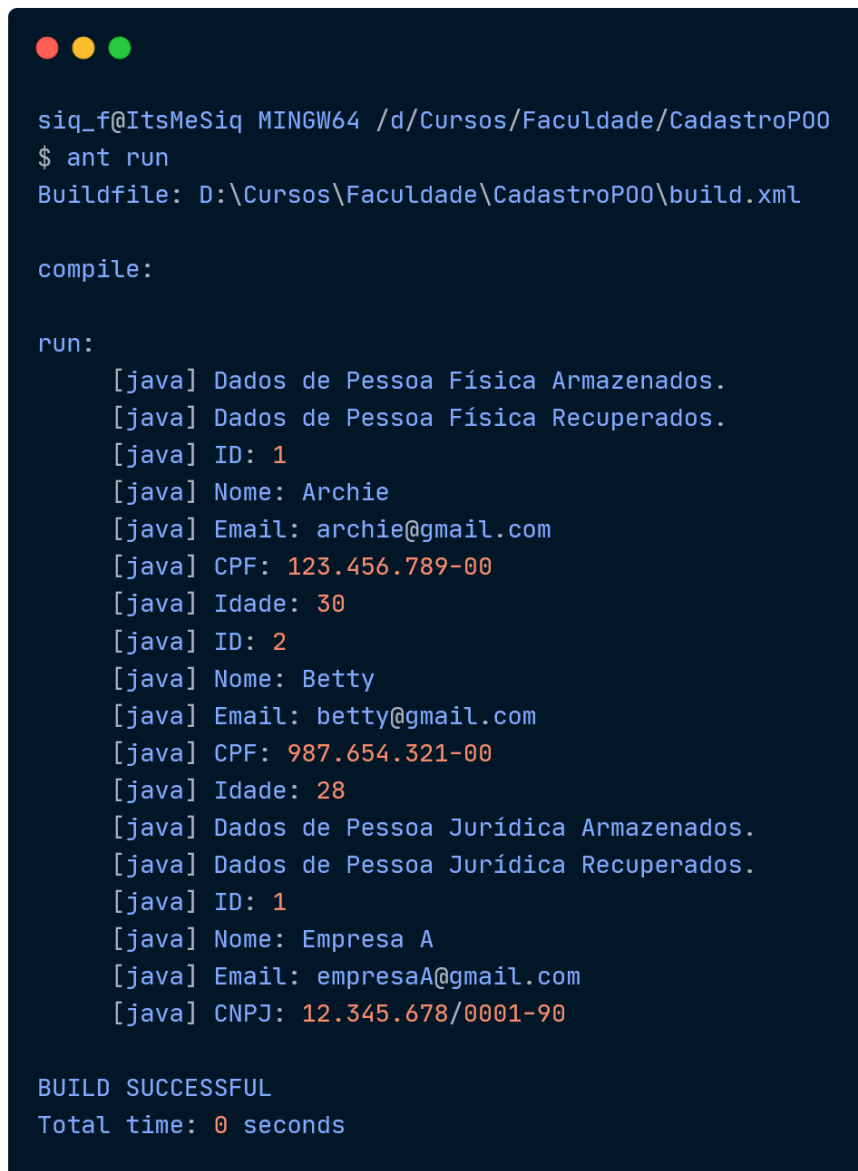
            PessoaJuridicaRepo repo4 = new PessoaJuridicaRepo();

            repo4.recuperar("pessoas-juridicas.bin");

            System.out.println("Dados de Pessoa Jurídica Recuperados.");

            for(PessoaJuridica empresa : repo4.obterTodos()) {
                empresa.exibir();
            }
        } catch (Exception e) {
            System.out.println("Erro: " + e.getMessage());
        }
    }
}
```

Figura 6 – Classe Main



```
siq_f@ItsMeSiq MINGW64 /d/Cursos/Faculdade/CadastroP00
$ ant run
Buildfile: D:\Cursos\Faculdade\CadastroP00\build.xml

compile:

run:
    [java] Dados de Pessoa Física Armazenados.
    [java] Dados de Pessoa Física Recuperados.
    [java] ID: 1
    [java] Nome: Archie
    [java] Email: archie@gmail.com
    [java] CPF: 123.456.789-00
    [java] Idade: 30
    [java] ID: 2
    [java] Nome: Betty
    [java] Email: betty@gmail.com
    [java] CPF: 987.654.321-00
    [java] Idade: 28
    [java] Dados de Pessoa Jurídica Armazenados.
    [java] Dados de Pessoa Jurídica Recuperados.
    [java] ID: 1
    [java] Nome: Empresa A
    [java] Email: empresaA@gmail.com
    [java] CNPJ: 12.345.678/0001-90

BUILD SUCCESSFUL
Total time: 0 seconds
```

Figura 7 – Resultado da Execução

3.3 Análise e Conclusão

3.3.1 Quais as vantagens e desvantagens do uso de herança?

A herança permite criar novas classes a partir de classes existentes, possibilitando o reaproveitamento de atributos e comportamentos. Além disso também existe a possibilidade de utilizar polimorfismo, permitindo que as classes derivadas forneçam diferentes implementações para o métodos existentes, utilizando o `@Override`.

Como desvantagem, o uso excessivo de herança pode aumentar o acoplamento entre as classes, fazendo com que alterações na classe-pai tenham impacto nas classes filhas. Por esse motivo, a herança deve ser utilizada quando existe uma relação adequada de especialização entre as classes.

3.3.2 Por que a interface `Serializable` é necessária ao efetuar persistência em arquivos binários?

A interface `Serializable` permite que os objetos sejam convertidos para uma representação que pode ser armazenada em arquivos binários, como arquivos com a extensão `.bin`. Dessa forma, os objetos presentes nos `ArrayList` puderam ser serializados e posteriormente reconstruídos durante a recuperação.

No projeto, `Pessoa` implementou `Serializable` e suas subclasses herdaram essa característica, permitindo a persistência das entidades.

3.3.3 Como o paradigma funcional é utilizado pela API `Stream` no Java?

A API `Stream` permite processar coleções utilizando operações funcionais, como filtragem, transformação e iteração, utilizando expressões `lambda`.

Por exemplo:

```
peessoas.stream().filter(p -> p.getId() > 1).forEach(p -> p.exibir());
```

Nesse exemplo, a coleção é transformada em um `Stream`, os elementos são filtrados por uma condição e, posteriormente, processados. Embora a implementação principal desta prática tenha utilizado diretamente `ArrayList`, o conceito de `Stream` está relacionado à abordagem funcional utilizada pelo Java para processamento de coleções.

3.3.4 Quando trabalhamos com Java, qual padrão de desenvolvimento é adotado na persistência de dados em arquivos?

No desenvolvimento Java, o padrão mais utilizado para organizar e isolar a persistência de dados em arquivos é o `DAO` (`Data Access Object`), frequentemente complementado pelo padrão `Repository`.

O padrão `DAO` tem como principal objetivo centralizar todas as operações de acesso a dados (como criar, ler, atualizar e excluir), separando completamente essa responsabilidade da lógica de negócio da aplicação. Já o padrão `Repository` atua de forma semelhante, mas com uma visão mais voltada para o domínio da aplicação, tratando o conjunto de dados como uma coleção de objetos.

No contexto do projeto, os repositórios `PessoaFisicaRepo` e `PessoaJuridicaRepo` foram responsáveis por encapsular toda a lógica de persistência. Eles funcionaram como uma implementação prática desses padrões, pois concentraram as operações de manipulação dos dados e esconderam os detalhes de leitura e escrita em arquivos binários. A persistência foi realizada por meio de serialização de objetos com `ObjectOutputStream` e `ObjectInputStream`, mas essa implementação ficou totalmente encapsulada dentro dos repositórios, garantindo o desacoplamento entre a regra de negócio e a camada de persistência.

O fluxo de funcionamento pode ser representado da seguinte forma:

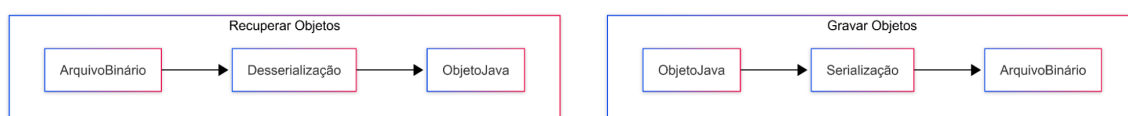


Figura 8 – Fluxograma Persistência de Arquivos